Online Examination Portal

System Design & Interview Notes

Java foundations - HashMap internals, Streams grouping
Building blocks - gateways, load balancing, rate limiting, storage strategy, disaster recovery, replication
Full design - concept level, then AWS, then Azure
Connectivity - how a browser reaches a private backend
Interview Q&A - 30+ questions with model answers

AI-generated content

Assembled by Claude (Anthropic) from a working session. Code is reasoned through but not executed against a live cluster. Capacity figures are illustrative estimates for discussion, not published data. Cloud service names, quotas and pricing change frequently - verify against current provider docs.

23 July 2026

Contents

Part 1 — Java foundations 1.1 Implementing a HashMap from scratch 1.2 Grouping a map by its values

Part 2 — Distributed systems building blocks 2.1 Single server, and why it fails 2.2 API Gateway vs Load Balancer 2.3 How load balancers actually work 2.4 Rate limiting 2.5 Storage strategy — where binary data belongs 2.6 Disaster recovery 2.7 Database replication

Part 3 — Online examination portal, concept level 3.1 Requirements 3.2 Capacity estimation 3.3 Level 1 — basic architecture 3.4 Level 2 — scaled architecture 3.5 Level 3 — detailed architecture 3.6 Data model 3.7 Critical flows

Part 4 — AWS implementation

Part 5 — Azure implementation

Part 6 — How the frontend reaches the backend over the internet

Part 7 — Interview questions and answers

Part 1 — Java foundations

1.1 Implementing a HashMap from scratch

Why this gets asked

It is the single densest interview question in Java. A complete answer touches array indexing, bit manipulation, the `equals`/`hashCode` contract, amortised complexity analysis, collision strategy, and — if you go far enough — thread safety and DoS resistance.

The five core ideas

Idea	Detail
Bucket array	Array of `Node` references. Capacity is always a power of two so index = `hash & (capacity - 1)` instead of a modulo — bitwise AND is far cheaper than division.
Hash spreading	User-supplied `hashCode()` often has poor low-bit distribution. JDK applies `h ^ (h >>> 16)` to mix high bits down so they influence bucket choice.
Collision resolution	Separate chaining — a linked list per bucket. JDK 8+ converts a bucket to a red-black tree past 8 nodes, capping worst case at O(log n).
Load factor	At 0.75 occupancy, double capacity and rehash. This is what keeps average operations O(1).
equals/hashCode contract	Compare `hash == hash && (k == key \ \ key.equals(k))`. The `==` short-circuit avoids a virtual call on the identity case.

Implementation

```
import java.util.Objects;

public class MyHashMap<K, V> {

    static final int   DEFAULT_CAPACITY    = 16;
    static final float DEFAULT_LOAD_FACTOR = 0.75f;

    static class Node<K, V> {
        final int hash;
        final K key;
        V value;
        Node<K, V> next;

        Node(int hash, K key, V value, Node<K, V> next) {
            this.hash = hash; this.key = key;
            this.value = value; this.next = next;
        }
    }

    private Node<K, V>[] table;
    private int size;
    private int threshold;
    private final float loadFactor;

    public MyHashMap() { this(DEFAULT_CAPACITY, DEFAULT_LOAD_FACTOR); }

    @SuppressWarnings("unchecked")
    public MyHashMap(int initialCapacity, float loadFactor) {
        if (initialCapacity <= 0) throw new IllegalArgumentException("capacity must be > 0");
        if (loadFactor <= 0 || Float.isNaN(loadFactor))
            throw new IllegalArgumentException("bad load factor");

        int cap = tableSizeFor(initialCapacity);
        this.loadFactor = loadFactor;
        this.table      = (Node<K, V>[]) new Node[cap];
        this.threshold  = (int) (cap * loadFactor);
    }

    /** Round up to the next power of two. */
```

```
    private static int tableSizeFor(int c) {
        int n = c - 1;
        n |= n >>> 1;  n |= n >>> 2;  n |= n >>> 4;
        n |= n >>> 8;  n |= n >>> 16;
        return (n < 0) ? 1 : n + 1;
    }

    /** XOR high 16 bits into low 16. Null key hashes to 0. */
    static int spread(Object key) {
        int h;
        return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
    }

    private int indexFor(int hash) {
        return hash & (table.length - 1);
    }

    public V put(K key, V value) {
        int hash = spread(key);
        int i    = indexFor(hash);

        for (Node<K, V> e = table[i]; e != null; e = e.next) {
            if (e.hash == hash && (e.key == key || Objects.equals(e.key, key))) {
                V old = e.value;
                e.value = value;              // update in place, size unchanged
                return old;
            }
        }

        table[i] = new Node<>(hash, key, value, table[i]);   // prepend
        if (++size > threshold) resize();
        return null;
    }

    public V get(Object key) {
        Node<K, V> e = getNode(key);
        return e == null ? null : e.value;
    }

    public boolean containsKey(Object key) { return getNode(key) != null; }

    private Node<K, V> getNode(Object key) {
        int hash = spread(key);
        for (Node<K, V> e = table[indexFor(hash)]; e != null; e = e.next) {
            if (e.hash == hash && (e.key == key || Objects.equals(e.key, key))) return e;
        }
        return null;
    }

    public V remove(Object key) {
        int hash = spread(key);
        int i    = indexFor(hash);

        Node<K, V> prev = null;
        for (Node<K, V> e = table[i]; e != null; prev = e, e = e.next) {
            if (e.hash == hash && (e.key == key || Objects.equals(e.key, key))) {
                if (prev == null) table[i] = e.next;
                else              prev.next = e.next;
                size--;
                return e.value;
            }
        }
        return null;
    }

    @SuppressWarnings("unchecked")
    private void resize() {
        Node<K, V>[] old = table;
        int newCap = old.length << 1;
        if (newCap <= 0) return;                     // overflow guard

        Node<K, V>[] next = (Node<K, V>[]) new Node[newCap];
        for (Node<K, V> head : old) {
            Node<K, V> e = head;
            while (e != null) {
                Node<K, V> nextNode = e.next;        // save before rewiring
                int i = e.hash & (newCap - 1);
                e.next  = next[i];
                next[i] = e;
                e = nextNode;
            }
        }
```

```
        table     = next;
        threshold = (int) (newCap * loadFactor);
    }

    public int     size()    { return size; }
    public boolean isEmpty() { return size == 0; }
}
```

Complexity

Operation	Average	Worst (chaining)	Worst (JDK 8+ treeified)
`get`	O(1)	O(n)	O(log n)
`put`	O(1) amortised	O(n)	O(log n)
`remove`	O(1)	O(n)	O(log n)
`resize`	O(n), amortised O(1) per insert	—	—

Follow-up points that separate a good answer from a great one

- **Treeification** — real `HashMap` converts a bucket to a red-black tree at 8 nodes and reverts at 6. The hysteresis prevents thrashing. The real motivation is DoS resistance: an attacker who can control keys could otherwise force every entry into one bucket.
- **Split-on-resize optimisation** — after doubling, an entry either stays at index `i` or moves to `i + oldCap`, determined by one bit test (`e.hash & oldCap`). JDK 8 exploits this to avoid recomputing hashes. The implementation above recomputes, which is clearer but slower.
- **Mutable keys** — if a key's `hashCode()` changes after insertion, the entry becomes unreachable. This is the bug behind "my map lost my entry."
- **Iteration order** — prepend-chaining means insertion order is not preserved. `LinkedHashMap` adds a doubly-linked list across all nodes to fix this.
- **Concurrency** — the pre-Java-8 resize could form a cycle in a bucket under concurrent writes, causing an infinite loop in `get()`. This is the historical reason `ConcurrentHashMap` exists.
- **The alternative design** — open addressing (linear probing, Robin Hood hashing). No per-entry node allocation, far better cache locality, but deletion needs tombstones and it degrades sharply above ~0.7 load. Most modern non-JVM hash maps use it.

1.2 Grouping a map by its values

Problem: given `Map<Integer, String>` of employee ID to department, produce `Map<String, List<Integer>>` of department to employee IDs.

Streams

```
Map<String, List<Integer>> byDept = empMap.entrySet().stream()
        .collect(Collectors.groupingBy(
                Map.Entry::getValue,                                  // classifier
                Collectors.mapping(Map.Entry::getKey, Collectors.toList())));
```

Without the `mapping` downstream you get `Map<String, List<Map.Entry<Integer,String>>>`, which is rarely useful.

Plain loop — faster, easier to debug

```
Map<String, List<Integer>> byDept = new HashMap<>();
empMap.forEach((empId, dept) ->
        byDept.computeIfAbsent(dept, k -> new ArrayList<>()).add(empId));
```

Variants

```
// Sorted departments
.collect(Collectors.groupingBy(Map.Entry::getValue, TreeMap::new,
         Collectors.mapping(Map.Entry::getKey, Collectors.toList())));
```

```
// Headcount per department -> Map<String, Long>
.collect(Collectors.groupingBy(Map.Entry::getValue, Collectors.counting()));

// IDs as a sorted set
Collectors.mapping(Map.Entry::getKey, Collectors.toCollection(TreeSet::new))
```

Gotchas

- `groupingBy` throws `NullPointerException` if any classifier value is `null`. Filter or wrap first.
- The returned map has no ordering guarantee unless you pass a map factory (`TreeMap::new`, `LinkedHashMap::new`).
- `Collectors.toMap` throws on duplicate keys unless you supply a merge function — a very common production bug.

Part 2 — Distributed systems building blocks

2.1 Single server, and why it fails

The canonical starting architecture: one machine running web server, application, database and cache. DNS resolves both `www.site.com` and `api.site.com` to its single IP.

Every failure mode lives here at once: one availability zone, one instance, one disk. If the instance dies you lose the application *and* the data. There is no way to deploy without downtime, no way to absorb a traffic spike, and no isolation between a runaway query and your web tier.

The reason textbooks start here is not that it is a good architecture. It is that each subsequent chapter removes exactly one of these failure modes, and a load balancer or a read replica is far easier to understand once you have felt the specific problem it solves.

The evolution path:

Step	What it removes
Separate the database onto its own host	Resource contention between app and DB
Add a load balancer and a second app instance	Single point of failure in the app tier; enables zero-downtime deploys
Add database read replicas	Read capacity ceiling
Add a cache (Redis)	Repeated identical DB reads
Add a CDN	Static asset load and global latency
Make the app tier stateless (move sessions to Redis)	Sticky-session constraints on scaling
Shard the database	Write capacity ceiling
Split into microservices	Team and deployment coupling
Add a message queue	Synchronous coupling between services
Multi-region	Regional failure

2.2 API Gateway vs Load Balancer

What an API Gateway does

A gateway is a **policy enforcement point**. Everything it does concerns the contract between a consumer and your APIs:

- Authentication and authorisation — validate JWTs, introspect OAuth tokens, check API keys, map claims to scopes
- Rate limiting and quotas — per consumer, per plan, per endpoint
- Routing — path, header, or version-based dispatch
- Request validation — reject malformed payloads before they reach a service
- Transformation — protocol translation, header injection, response shaping
- Caching for idempotent reads
- Per-consumer observability, which is what makes "who is hammering us" an answerable question
- Monetisation and developer portal (APIGEE's differentiator over open-source gateways)

The unifying theme: **a gateway knows who the caller is and what they are allowed to do.**

The distinction that matters

	Load balancer	API Gateway
Decides	*Which instance* of a service	*Which service*, and *whether you're allowed*
Knows about	Connections, health, latency	Identity, contracts, quotas, plans
State held	Connection pools, health status	Tokens, rate-limit counters, API definitions
Failure mode	Traffic sent to a dead box	Unauthorised access, quota abuse

	Load balancer	API Gateway
OSI layer	4 or 7	7 only

Does an API Gateway do load balancing?

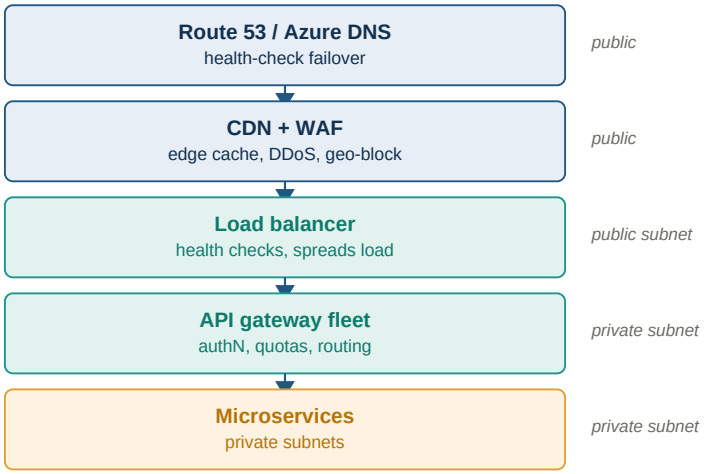
Incidentally, yes — most gateways round-robin across upstream instances. But that is a side effect, not the job.

The clinching argument: the gateway itself is a fleet. If you run five Spring Cloud Gateway instances, something must distribute traffic across those five. A gateway cannot load balance itself.

Do you need a separate load balancer?

Gateway type	Separate LB needed?	Why
Amazon API Gateway, Azure API Management (consumption tier)	**No**	Managed, serverless, multi-AZ, scales automatically. It *is* the entry point.
Spring Cloud Gateway, Kong, APIGEE hybrid, self-hosted anything	**Yes**	You are running N JVM processes. They need health checks, TLS termination, and AZ distribution in front.
Kubernetes Ingress controller	Collapsed	The Ingress controller plays both roles. Both responsibilities still exist, they are just co-located.

Typical layering for a self-hosted gateway:



Layering when the gateway is self-hosted

2.3 How load balancers actually work

Layer 4 vs Layer 7

	L4 (transport)	L7 (application)
Operates on	TCP connections	HTTP requests
Sees	IP, port	Host, path, headers, method, body
Can route by path?	No	Yes
TLS termination	Pass-through or TLS-level	Full termination and re-encryption
Source IP	Preserved	Replaced (see `X-Forwarded-For`)
Throughput	Millions of rps	Lower per-request, far more capable
AWS / Azure	NLB / Azure Load Balancer	ALB / Application Gateway

Selection algorithms

Algorithm	How it works	When to use
Round robin	Next instance in sequence	Uniform request cost, uniform instances
Weighted round robin	Proportional to assigned weight	Heterogeneous instance sizes, canary rollouts
Least connections	Fewest open connections wins	Long-lived connections
Least outstanding requests	Fewest in-flight requests wins	Highly variable request cost. Usually the better ALB setting.
Consistent hashing	Same key always maps to same instance	Cache locality, session affinity. Adding an instance remaps only 1/n of keys.
Power of two choices	Pick two at random, send to the less loaded	Near-optimal with zero coordination. What most service meshes actually do.

Health checking

Active — probe an endpoint every N seconds. Simple, but there is a detection window during which traffic still goes to a dead instance. Tune `interval × unhealthyThreshold` against your error budget.

Passive (outlier detection) — watch real traffic and eject an instance after consecutive 5xx responses. Instant, but can cascade. Always cap the maximum ejection percentage so a widespread backend fault does not eject your entire fleet.

Use both. Active catches an instance that has stopped listening; passive catches one that is accepting connections but returning garbage.

Connection draining — the part people forget

When you deregister an instance, the balancer must stop sending *new* requests while allowing in-flight ones to complete. Skip this and every deployment drops requests.

- **ALB** — deregistration delay (default 300s, usually tune to 30s)
- **Kubernetes** — readiness probe must fail *before* the pod stops accepting connections; combine `preStop` sleep with `terminationGracePeriodSeconds`

Other essentials

- **Cross-zone load balancing** — without it, an AZ with fewer instances receives the same share of traffic and each instance there is overloaded. Enable it.
- **Sticky sessions** — available via cookies, but prefer stateless services with shared session state in Redis. Stickiness undermines even distribution and complicates deploys.
- **Slow start** — ramp traffic to a newly registered instance so JIT warm-up and connection pools have time to establish.

2.4 Rate limiting

The five algorithms

Algorithm	Bursts	Memory	Weakness
Fixed window counter	Allows 2× at window boundaries	O(1)	The boundary burst is a genuine exploit
Sliding window log	Exact	O(n) timestamps per key	Memory blows up under load
Sliding window counter	Approximate, weighted	O(1)	Slight inaccuracy, usually acceptable
Token bucket	Configurable burst capacity	O(1)	The usual right answer
Leaky bucket	None — smooths output to constant rate	O(1) + queue	Adds latency, no burst tolerance

Token bucket in one line: tokens refill at a steady rate, bucket capacity defines the burst you tolerate, each request consumes one token, empty bucket means reject.

Distributed implementation — Redis + Lua

The critical detail is **atomicity**. Read-modify-write across separate Redis round trips is a race: two concurrent requests both read "1 token left" and both proceed. A Lua script executes atomically inside Redis.

```
local key      = KEYS[1]
local capacity = tonumber(ARGV[1])
local rate     = tonumber(ARGV[2])   -- tokens per second
local now      = tonumber(ARGV[3])   -- epoch millis
local cost     = tonumber(ARGV[4])

local b      = redis.call('HMGET', key, 'tokens', 'ts')
local tokens = tonumber(b[1])
local ts     = tonumber(b[2])

if tokens == nil then
  tokens = capacity
  ts     = now
end

local elapsed = math.max(0, now - ts) / 1000
tokens = math.min(capacity, tokens + elapsed * rate)

local allowed = 0
if tokens >= cost then
  tokens  = tokens - cost
  allowed = 1
end

redis.call('HMSET', key, 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', key, math.ceil(capacity / rate * 2000))

return { allowed, math.floor(tokens) }
```

The `PEXPIRE` is not optional — without it you accumulate one key per client forever and eventually exhaust Redis memory.

Spring WebFlux filter

```
@Component
public class RateLimitFilter implements WebFilter {

    private final ReactiveStringRedisTemplate redis;
    private final RedisScript<List> script;   // loads the Lua above

    @Override
    public Mono<Void> filter(ServerWebExchange ex, WebFilterChain chain) {
        String key = "rl:" + resolveClientId(ex);   // API key > user id > IP

        return redis.execute(script, List.of(key),
                        List.of("100", "10",
                                String.valueOf(System.currentTimeMillis()), "1"))
                .next()
                .flatMap(res -> {
                    boolean allowed = ((Number) res.get(0)).intValue() == 1;
                    ex.getResponse().getHeaders()
                      .add("X-RateLimit-Remaining", String.valueOf(res.get(1)));

                    if (!allowed) {
                        ex.getResponse().setStatusCode(HttpStatus.TOO_MANY_REQUESTS);
                        ex.getResponse().getHeaders().add("Retry-After", "1");
                        return ex.getResponse().setComplete();
                    }
                    return chain.filter(ex);
                })
                .onErrorResume(e -> chain.filter(ex));   // FAIL OPEN
    }
}
```

The decisions that actually matter

Fail open, not closed. That `onErrorResume` is deliberate. If Redis blips you do not want the rate limiter to become an outage. The opposite applies to authentication — auth fails *closed*. Getting these backwards is a classic incident.

Redis is now on the hot path of every request. That is a latency tax and a shared dependency. The standard mitigation is a hybrid: each instance holds a local bucket for a share of the quota and syncs with Redis periodically. You trade exactness for latency, which is almost always right for rate limiting.

Choose the key carefully. API key is best, authenticated user ID next. IP is weakest — carrier-grade NAT means thousands of mobile users share one address and blocking it removes them all. For IPv6, limit on the `/64` prefix, not the individual address.

Return the right response. HTTP `429`, plus `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`, and `Retry-After`. Clients given no information retry immediately and make things worse.

Do not build this unless you must. Spring Cloud Gateway ships `RequestRateLimiter` with exactly this Redis token bucket. Bucket4j is a mature Java implementation with distributed backends. AWS WAF rate-based rules and Azure API Management throttling policies handle the crude volumetric case at the edge. Understanding the algorithm is essential; hand-rolling the implementation usually is not.

2.5 Storage strategy — where binary data belongs

The rule

Never store photos, signatures, or documents in the database. Store them in object storage and keep the key plus metadata in the database.

Why

Problem	Consequence
Backup size	A 400 GB database backs up and restores in hours, not minutes
Replication	Every read replica carries every image, multiplying storage cost
Row size limits	PostgreSQL TOASTs large values into side tables; performance degrades
Cost	Block storage is roughly 5× the price per GB of object storage
CDN	You cannot serve a database BLOB through a CDN
Connection pool	Streaming large objects occupies a connection for the duration

At 1.5M candidates × 2 images × 200 KB, that is 600 GB of binary in a database that should be a few tens of GB.

The correct split

```
Object storage (S3 / Blob Storage)     Database (PostgreSQL)
------------------------------         ---------------------
2026/a3/1048576/photo.jpg      <----   s3_key, version_id, sha256,
2026/a3/1048576/signature.jpg          content_type, size, status
```

Upload pattern — presigned URLs

Never route uploads through your application servers. At scale that is hundreds of GB occupying request threads for no reason.

```
1. Browser asks API for an upload URL
2. API returns a presigned URL, scoped to one key, one content type,
   a max content-length, expiring in ~5 minutes
3. Browser PUTs the file directly to object storage
4. Object-created event triggers a function
5. Function validates, scans, thumbnails, writes status to the DB
```

The presigned URL is scoped so tightly that a candidate cannot overwrite another candidate's object even by tampering with the request.

Object store settings that are non-negotiable

- **Block all public access.** Serve through the CDN with origin access control and signed URLs. A candidate's photo must never be fetchable by guessing a URL.
- **Versioning plus object lock (compliance mode)** for the statutory retention period. When a result is challenged in court you must prove the photo on the admit card is the photo that was uploaded.
- **Customer-managed encryption keys** rather than provider-default — the key policy becomes a second access-control layer and you get decrypt events in the audit log.
- **Lifecycle policy:** hot tier -> infrequent access at 90 days -> archive after results -> delete at end of retention.
- **Separate bucket per environment.** Never let a test job point at production PII.

Validation at upload, not at the exam centre

This is where large exams actually fail. A candidate uploads a blurry group photo in March; nobody notices until an invigilator cannot match a face in May.

Check	Mechanism
Format and size	Read magic bytes, not the file extension. Reject non-JPEG/PNG, over 300 KB, wrong dimensions.
Face detection	Exactly one face, eyes open, frontal pose, no sunglasses, adequate brightness
Signature validity	Aspect ratio ~3:1, ink coverage between thresholds (catches blank scans and scribbles), predominantly white background
Malware	Managed malware scanning on the bucket, or ClamAV in the function
Duplicate faces	Face collection search — flags the same face registering under multiple application numbers

Push failures back to the candidate immediately with a specific reason. Every rejection caught in March is a support ticket and a centre-day dispute you do not have in May.

2.6 Disaster recovery

Everything follows from two numbers agreed **before** choosing any service:

- **RPO (Recovery Point Objective)** — how much data you can afford to lose, measured in time
- **RTO (Recovery Time Objective)** — how long you can be down

A stakeholder who says "zero and zero" is saying "unlimited budget." The conversation has to force a real number.

The four strategies

Strategy	RPO	RTO	Standby cost	Running in DR region
Backup and restore	Hours	12–24h+	~5%	Nothing but backups
Pilot light	Minutes	Tens of minutes to hours	~15%	Data replicating; compute defined but off
Warm standby	Seconds	Minutes	~50%	Full stack, scaled down
Multi-site active/active	Near zero	Near zero	~200%	Full stack, serving traffic

The jump from warm standby to active/active is not incremental. It forces you to solve multi-region write conflicts, which is an application design problem, not an infrastructure one.

What actually goes wrong

- **Multi-AZ is not DR.** It protects against a datacentre failure. It does not protect against a regional control-plane outage, a bad deploy, an accidental `DROP TABLE`, or a compromised account.
- **Service quotas are per-region and default low.** Your DR region will happily replicate data for two years and then refuse to launch 200 instances at the moment of truth. Raise limits proactively and test at full scale.
- **Encryption keys are regional.** Cross-region encrypted snapshot copies need a key in the destination region with correct grants. This is the single most common first-real-test failure.
- **Failback is harder than failover** and almost nobody plans it. Once you are running in DR with newer data, returning means reversing replication and taking a second outage.
- **An untested plan is not a plan.** Run scheduled game days where you actually fail over. The failure modes you find are never the ones you designed for.

Practical sequence

1. Write down RPO/RTO **per workload** — they are not uniform. A reporting dashboard and a transaction ledger deserve different answers.
2. Start with backup and restore everywhere. Verify restores work, on a schedule.
3. Upgrade only the tiers whose business impact justifies it.
4. Automate failover in code, not a runbook wiki page.
5. Test quarterly. Measure actual RTO against the promised one.

2.7 Database replication

The three mechanisms underneath every managed option

Snapshot / backup copy — periodic full or incremental copies shipped cross-region. RPO equals your snapshot interval, usually hours. Cheapest, slowest recovery.

Log shipping / physical replication — the storage layer or write-ahead log stream is continuously sent and applied. RPO in seconds. A standby cluster exists and consumes resources.

Multi-primary with conflict resolution — both regions accept writes, replication is bidirectional, conflicts resolved by rule (usually last-writer-wins). RPO near zero, but you inherit a correctness problem the application must handle.

Options by engine

Engine	Mechanism	Typical RPO	Promotion
Aurora Global Database	Storage-layer	Sub-second	Managed failover (~1 min) or detach-and-promote
RDS cross-region read replica	Async engine-level	Seconds to minutes	Manual promote — **breaks replication permanently**
RDS cross-region backup replication	Automated backup copy	Minutes	Restore to a new instance
DocumentDB global cluster	Storage-layer	~1 second	Manual promote of a secondary
DynamoDB Global Tables	Multi-active, LWW	Sub-second	None needed — already live
MongoDB replica set (Atlas)	Oplog	Milliseconds	Automatic election
Azure SQL active geo-replication	Log shipping	Seconds	Manual or auto-failover group
Cosmos DB multi-region writes	Multi-master	Sub-second	Automatic

MongoDB specifics

Multi-region replica set is the cleanest option. Place a 3-node set as, say, 2 nodes in Mumbai and 1 in Chennai. Replication is the normal oplog stream. Lose the primary region and the survivors hold an election.

The trade-off: `writeConcern: "majority"` with a cross-region majority means every write pays inter-region latency. At ~35 ms RTT, that is your write floor. Most teams keep the voting majority in the primary region and treat the remote member as DR-only (`priority: 0`, `votes: 0`) — cheap writes, but failover becomes a manual reconfiguration rather than an automatic election. **That is a deliberate RTO trade, and you should be able to state it explicitly in an interview.**

The parts people get wrong

- **Replication lag is your real RPO** — not the marketing number. Instrument it, alarm on it, and know what it looks like during peak write volume. A bulk migration or a large index build will blow it out.
- **Promotion is usually one-way.** A promoted RDS read replica is a separate database forever.
- **Split-brain during failover.** If the primary region is partitioned rather than dead, both regions may accept writes. Fencing matters — for MongoDB that is what the voting configuration does; for everything else you need an external arbiter of truth.
- **Schema and index changes must replicate too.** DDL not captured by your replication mechanism makes the DR region silently divergent.

Part 3 — Online examination portal, concept level

3.1 Requirements

Functional

Area	Requirement
Registration	Candidate signup, profile, photo and signature upload, document verification
Payment	Fee collection, reconciliation, refund on rejected application
Exam slot	City preference, centre allocation, shift assignment
Admit card	Generated after verification, downloadable, contains photo and barcode
Question bank	Multiple question types (MCQ single, MCQ multiple, numerical, matrix match, comprehension) with versioning
Exam delivery	Timed exam, question navigation, mark-for-review, autosave, section locking
Proctoring	Identity verification at centre, webcam capture, activity logging
Submission	Final submit, auto-submit on timer expiry, response sheet generation
Evaluation	Answer key publication, objection window, normalisation across shifts, result computation
Results	Score card, rank, category-wise cutoffs

Non-functional — these drive the architecture

Attribute	Target	Why it matters
Availability during exam	99.99% within the exam window	A 5-minute outage during a shift means re-conducting the exam for those candidates. This is a national news event.
Data durability	Zero tolerance for lost answers	A lost response is a legal liability, not a bug
Latency	p99 < 200 ms for answer save	Candidates perceive lag as lost time and file grievances
Consistency	Strong for responses, eventual acceptable for analytics	A candidate must always see their own last saved answer
Integrity	Tamper-evident audit trail	Results are challenged in court routinely
Fairness	Identical experience across shifts	Normalisation depends on comparable conditions
Security	Question paper confidentiality until exam start	Leak means cancellation
Compliance	DPDP Act 2023, data residency in India	Legal requirement

The defining characteristic

This is a **spiky, deadline-driven workload with an absolute correctness requirement.** It is not a social network where losing a like is acceptable. Design decisions should consistently favour durability and auditability over throughput and cost.

3.2 Capacity estimation

Illustrative figures for design discussion, modelled on a national entrance exam.

Registration phase

Metric	Value	Derivation
Total candidates	1.5 M	Given
Registration window	30 days	Given
Traffic distribution	~40% in final 72 hours	Universal pattern for deadline-driven systems
Average registration rate	~0.6/s	1.5M / (30 × 86400)
Peak registration rate	~200/s	600K over 3 days, concentrated in ~8 waking hours

Metric	Value	Derivation
Document uploads	3 M objects	1.5 M × 2 (photo + signature)
Upload storage	~600 GB	3 M × 200 KB

Exam phase

Metric	Value	Derivation
Shifts	12	2 per day × 6 days
Candidates per shift	~125,000	1.5 M / 12
Exam duration	3 hours	Given
Questions per paper	90	Given
Autosave interval	20 s	Design choice
Sustained write rate	**~6,250 writes/s**	125,000 / 20
Writes per shift	~67 M	125,000 × (10800 / 20)
Write payload	~200 bytes	Candidate ID, question ID, response, timestamp, state
Question fetch reads	11.25 M per shift	125,000 × 90 — but prefetched and cached, so near-zero DB load

Results phase

Metric	Value	Note
Candidates checking results	1.5 M	
Concentration	~60% within first 2 hours	
Peak read rate	**~125,000 req/s** if served dynamically	This is why results are pre-rendered to object storage and served from the CDN — the database sees almost nothing

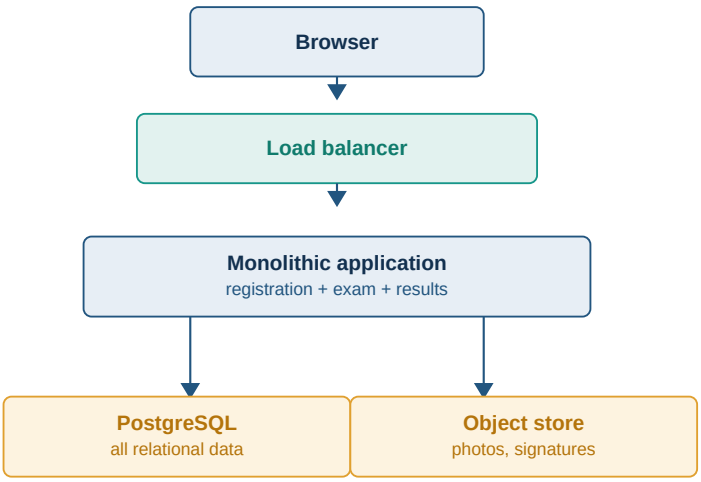
Storage totals

Data	Size
Candidate records	~3 GB (1.5 M × 2 KB)
Documents	~600 GB
Exam responses	~160 GB (67 M × 12 shifts × 200 B)
Audit and event logs	~1–2 TB
Question bank	< 1 GB

The headline insight for an interview: the write rate during the exam (~6,250/s) is modest for a modern datastore. The hard parts are not throughput — they are **durability under failure, the results-day read spike, and the fact that you cannot retry the exam.**

3.3 Level 1 — basic architecture

The minimum viable shape. Suitable for a college with a few thousand candidates.



Level 1 - basic single-application architecture

Properties: simple to build and reason about; a single deployable; strong consistency for free.

Where it breaks: the results-day read spike takes down the exam service because they share a database. There is no isolation between the low-stakes registration workload and the zero-tolerance exam workload. Deployments during the exam window are impossible.

3.4 Level 2 — scaled architecture

Introduce caching, read replicas, a CDN, and separate the exam-delivery path.

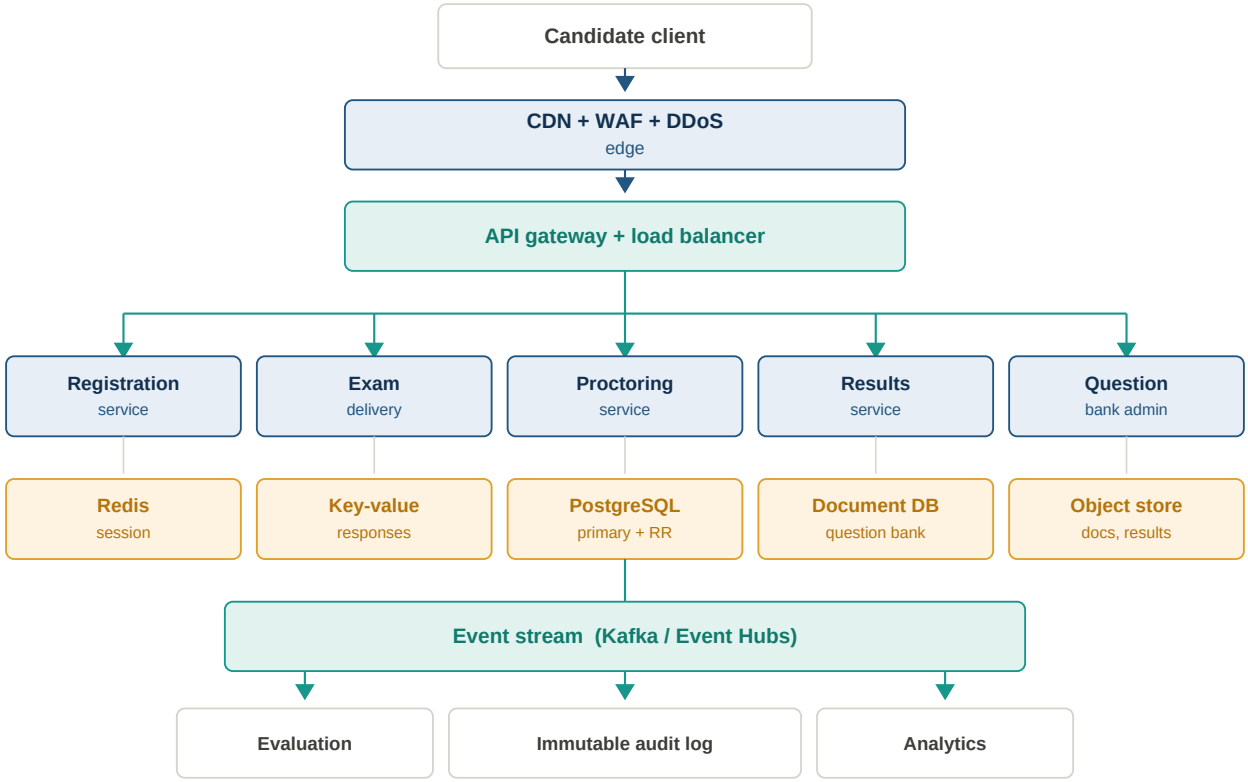
```
        Browser
           |
       CDN  (static assets, results)
           |
     WAF + API Gateway
           |
       Load balancer
           |
   +-------+--------+---------+
   |       |        |         |
Registration  Exam   Results  Admin
  service   service  service  service
   |          |         |
   |     +----+----+    |
   |     |         |    |
   |   Redis    Answer  |
   |  (session)  store  |
   |                    |
   +-----+--------+-----+
         |        |
    PostgreSQL  Read
    (primary)   replicas
```

Key changes and why:

- **Exam service is isolated** with its own datastore. Nothing that happens to registration or results can affect a live exam.
- **Redis holds exam session state** — timer, current question, navigation state. Sub-millisecond reads, and it takes the load off the primary database.
- **Read replicas** absorb the results-day read volume.
- **CDN** serves static assets, and critically, the pre-rendered result pages.
- **API Gateway** centralises authentication and rate limiting so services do not each reimplement it.

3.5 Level 3 — detailed architecture

Production shape for national scale.



Level 3 - production architecture at national scale

Design decisions worth defending in an interview

1. The event stream is the source of truth for exam responses, not the database.

Every answer save is published to a partitioned log before or alongside the database write. The database becomes a materialised projection. This changes disaster recovery from "hope replication lag was low" to "replay from offset." For a system where losing a response is a legal liability, that is a fundamentally stronger guarantee.

The cost: consumers must be genuinely idempotent, and offsets do not translate cleanly across replicated clusters — you need offset translation or timestamp-based seek.

2. Question bank is a document store; candidate registration is relational.

Question types are genuinely heterogeneous — an MCQ, a numerical-answer question, and a matrix-match question have different shapes. That is a real document-model fit.

Registration is the opposite: payment confirmation, application-number generation and slot allocation must be atomic, and duplicate-registration prevention wants a real unique constraint. **Push back if an interviewer suggests a document store for candidate identity data.**

3. Answer storage is a separate high-throughput key-value store.

The access pattern is a single-partition read/write keyed by `(candidate_id, exam_session_id)`. No joins, no cross-candidate queries during the exam. That is exactly the shape a key-value store is optimised for, and it keeps 6,250 writes/s off the relational primary.

4. Results are pre-rendered, not computed on request.

Evaluation runs as a batch job after the objection window closes. Each candidate's score card is written as a static object and served from the CDN. The 125,000 req/s spike hits edge caches; the database sees essentially nothing.

5. Audit log is append-only and immutable.

Every state transition — registration, verification, exam start, each answer save, submission, evaluation — is written to WORM storage with object lock. This is not observability. It is legal evidence.

3.6 Data model

Relational (candidate identity, payments, allocation)

```
CREATE TABLE candidates (
    candidate_id     BIGSERIAL PRIMARY KEY,
    application_no   VARCHAR(20)  UNIQUE NOT NULL,
    full_name        VARCHAR(150) NOT NULL,
    dob              DATE         NOT NULL,
    category         VARCHAR(20)  NOT NULL,
    exam_year        SMALLINT     NOT NULL,
    status           VARCHAR(20)  NOT NULL,   -- DRAFT|SUBMITTED|VERIFIED|REJECTED
    created_at       TIMESTAMPTZ  DEFAULT now()
);

CREATE TABLE candidate_documents (
    doc_id           BIGSERIAL PRIMARY KEY,
    candidate_id     BIGINT       NOT NULL REFERENCES candidates,
    doc_type         VARCHAR(20)  NOT NULL,   -- PHOTO|SIGNATURE|ID_PROOF
    bucket           VARCHAR(63)  NOT NULL,
    object_key       VARCHAR(512) NOT NULL,
    version_id       VARCHAR(64),             -- pins the exact object version
    content_type     VARCHAR(50),
    size_bytes       INT,
    sha256           CHAR(64),                -- integrity + duplicate detection
    status           VARCHAR(20)  NOT NULL,   -- PENDING|VALID|REJECTED
    rejection_reason TEXT,
    uploaded_at      TIMESTAMPTZ  DEFAULT now(),
    UNIQUE (candidate_id, doc_type)
);

CREATE TABLE exam_sessions (
    session_id       UUID PRIMARY KEY,
    candidate_id     BIGINT      NOT NULL REFERENCES candidates,
    shift_id         INT         NOT NULL,
    centre_id        INT         NOT NULL,
    paper_version    INT         NOT NULL,
    started_at       TIMESTAMPTZ,
    submitted_at     TIMESTAMPTZ,
    submit_reason    VARCHAR(20),             -- MANUAL|TIMEOUT|ADMIN
    UNIQUE (candidate_id, shift_id)
);
```

The `sha256` earns its place twice: it proves the object was not swapped after verification, and it catches thousands of candidates uploading an identical stock image — a real fraud pattern.

The `version_id` matters because if a candidate may re-upload a corrected photo, you still need to prove which version appeared on the admit card.

Key-value (exam responses)

```
Partition key:  session_id
Sort key:       question_id

Attributes:
  response        (string or array)
  state           NOT_VISITED | NOT_ANSWERED | ANSWERED
                  | MARKED_REVIEW | ANSWERED_MARKED_REVIEW
  updated_at      epoch millis
  client_seq      monotonic counter from the client
  server_seq      assigned on write
```

`client_seq` is the mechanism for **last-write-wins with out-of-order delivery**. If a delayed retry arrives carrying an older `client_seq`, it is rejected rather than overwriting a newer answer. Without this, a network hiccup can silently revert a candidate's answer — one of the most damaging bugs possible in this domain.

Document (question bank)

```
{
  "questionId": "Q-2026-PHY-00417",
  "paperVersion": 3,
  "subject": "PHYSICS",
  "section": "A",
  "type": "MCQ_SINGLE",
  "marks": 4,
  "negativeMarks": -1,
  "body": { "en": "...", "hi": "..." },
  "assets": ["s3://qb/2026/phy/00417-fig1.png"],
  "options": [
    { "id": "A", "text": { "en": "...", "hi": "..." } }
  ],
```

```
  "answerKey": "B",
  "keyPublishedAt": null
}
```

The `answerKey` field lives in a **separate, differently-permissioned collection** in production. Nobody with read access to the delivered question bank should have read access to the keys before publication.

3.7 Critical flows

Registration

```
1. Candidate creates account, verifies mobile/email OTP
2. Fills the form -> saved as DRAFT (idempotent, resumable)
3. Requests presigned upload URLs for photo + signature
4. Uploads directly to object storage
5. Async validation: format, dimensions, face detection, malware, duplicate face
6. On validation failure -> specific reason surfaced immediately
7. Payment -> gateway callback -> application number generated
   (This step must be atomic and idempotent: payment gateways
    retry callbacks, and a duplicate application number is unrecoverable)
8. Status -> SUBMITTED, admit card generated after verification
```

Exam delivery — the hard part

```
1. Candidate authenticates at centre; biometric or photo match against
   the stored image
2. Server issues a short-lived, session-bound exam token
3. Client prefetches the ENTIRE paper (encrypted) before the exam starts
   -> decryption key released only at the shift start time
   -> removes network dependency for question fetch during the exam
4. Timer is SERVER-authoritative. The client displays a countdown but
   the server holds the truth. Never trust the client clock.
5. Every answer change:
      a. write to IndexedDB locally (survives browser crash)
      b. POST to /responses with client_seq
      c. server writes to key-value store + publishes to event stream
      d. ack returns server_seq
6. Heartbeat every 10s carries current server time and remaining seconds
7. On disconnect: client keeps working from local cache, queues writes,
   syncs on reconnect. Elapsed time still counts down server-side.
8. Submission: manual, or auto-submit at expiry, or admin-forced
   -> response sheet snapshot written to WORM storage immediately
   -> candidate shown a read-only confirmation with a hash
```

The three things that make this design defensible:

- **Prefetch the paper.** Network failure mid-exam then costs the candidate nothing. This is the single highest-value decision in the whole design.
- **Server-authoritative timer.** Otherwise clock manipulation is trivial extra time.
- **Local-first writes with sequence numbers.** The candidate never loses an answer to a transient network fault, and out-of-order retries cannot revert newer answers.

Results

```
1. Answer keys published; objection window opens (typically 3 days)
2. Objections reviewed; keys revised where upheld
3. Evaluation batch job:
      raw score -> normalisation across shifts -> rank -> category cutoffs
4. Each score card rendered to a static object, signed
5. CDN warmed BEFORE the announced time
6. Announcement -> 125,000 req/s hits edge caches
   -> origin and database see near-zero load
```

Normalisation across shifts is worth understanding conceptually: because different shifts get different papers, raw scores are not directly comparable. Percentile-based normalisation maps each candidate's raw score to a percentile within their own shift, then scores are compared across shifts on the percentile scale. Any interviewer with domain knowledge will probe this.

Part 4 — AWS implementation

4.1 Service mapping

Layer	Service	Configuration notes
DNS	**Route 53**	Health-check failover routing; alias records to CloudFront
CDN + edge	**CloudFront**	OAC to S3, custom origin to ALB, cache policies per path
WAF	**AWS WAF**	Attach to CloudFront, not the ALB — block at the edge
DDoS	**Shield Advanced**	Worth the cost for a national exam; includes DRT support
API gateway	**API Gateway HTTP API** or self-hosted **Spring Cloud Gateway** on ECS	See cost note below
Load balancer	**ALB**	Least-outstanding-requests, cross-zone on, 30s deregistration delay
Compute	**ECS Fargate** (or EKS)	Fargate removes node management during the exam window
Serverless	**Lambda**	Document validation, presigned URL issuance, evaluation steps
Relational	**Aurora PostgreSQL**	Multi-AZ, 2+ read replicas, Global Database for DR
Document	**DocumentDB**	Question bank
Key-value	**DynamoDB**	Exam responses. On-demand capacity for the shift spike.
Cache	**ElastiCache for Redis**	Session state, rate-limit counters. Cluster mode enabled.
Object storage	**S3**	Documents, response sheets, pre-rendered results
Streaming	**MSK** (or Kinesis Data Streams)	Answer event log
Queue	**SQS**	Async jobs, DLQ for failed validations
Identity	**Cognito**	Candidate auth; or a custom OIDC provider if requirements are unusual
Secrets	**Secrets Manager**	DB credentials with automatic rotation
Keys	**KMS**	Customer-managed keys, separate key per data classification
Face matching	**Rekognition**	`DetectFaces`, `CompareFaces`, face collections for duplicate detection
Malware scan	**GuardDuty Malware Protection for S3**	
Monitoring	**CloudWatch**	
Tracing	**X-Ray** or **OpenTelemetry -> AMP/AMG**	
IaC	**CDK** or **Terraform**	
Region	**ap-south-1 (Mumbai)** primary, **ap-south-2 (Hyderabad)** DR	Both Indian regions — keeps data residency simple

4.2 Network topology

```
VPC 10.0.0.0/16, three availability zones

Public subnets      10.0.0.0/24, 10.0.1.0/24, 10.0.2.0/24
    -> ALB only. Internet gateway route.
    -> NAT gateway per AZ (one per AZ, not one shared — AZ isolation)

Private app subnets 10.0.10.0/24, 10.0.11.0/24, 10.0.12.0/24
    -> ECS tasks. Route to NAT for egress. No inbound from internet.

Isolated data subnets 10.0.20.0/24, 10.0.21.0/24, 10.0.22.0/24
    -> Aurora, DocumentDB, ElastiCache
    -> NO route to NAT or IGW. Completely unreachable from the internet.
```

```
VPC endpoints (Gateway):    S3, DynamoDB
VPC endpoints (Interface):  Secrets Manager, KMS, ECR, CloudWatch Logs,
                            SQS, Rekognition
    -> Traffic to these services never traverses the public internet
```

Security groups reference each other, never CIDR blocks.

```
sg-alb       inbound 443 from 0.0.0.0/0
sg-app       inbound 8080 from sg-alb          <- not a CIDR
sg-aurora    inbound 5432 from sg-app          <- not a CIDR
sg-redis     inbound 6379 from sg-app
```

This is the single most valuable habit in AWS networking. If an instance moves subnets, the rule still works. If someone adds a new app instance to `sg-app`, database access follows automatically — and nothing outside that group can reach the database regardless of its IP.

4.3 The API Gateway cost decision

Option	Cost per million requests	When to choose
API Gateway REST API	~$3.50	Rich features: usage plans, API keys, request validation, WAF integration
API Gateway HTTP API	~$1.00	Same core routing, ~70% cheaper, fewer features
ALB + self-hosted gateway	~$0.20 effective at high volume	Above a few hundred million requests/month

At 1 billion requests/month: REST API ~ $3,500, ALB ~ $200. **The crossover is roughly a few hundred million requests per month**, above which the operational cost of running Spring Cloud Gateway yourself is less than the difference.

For this workload the managed option wins — traffic is extremely spiky (idle for weeks, then a 12-day burst), and you do not pay for idle capacity.

4.4 Disaster recovery configuration

Component	Strategy	RPO	RTO
Aurora PostgreSQL	Global Database to ap-south-2	Sub-second	~1 min (managed failover)
DynamoDB	Global Tables	Sub-second	0 (already active)
S3	Cross-Region Replication + versioning + object lock	Minutes	0
DocumentDB	Global cluster	~1 s	Manual promote
MSK	MSK Replicator	Seconds	Consumer replay required
ECS	Pilot light — task definitions deployed, service at 0 desired count	N/A	Minutes to scale up
Route 53	Health-check failover, or ARC routing controls	N/A	DNS TTL bound

During the exam window specifically, run warm standby, not pilot light. The cost of a scaled-down second region for 12 days is trivial against the cost of re-conducting a national exam.

4.5 Exam-day operational checklist

- Pre-scale everything. Do not rely on autoscaling to react to a shift start — the ramp is instantaneous and autoscaling is not.
- DynamoDB on-demand, or provisioned with pre-warmed capacity.
- Aurora read replicas scaled up the day before.
- CloudFront cache pre-warmed for static assets.
- Service quotas verified: ECS tasks, ENIs, Lambda concurrency, NAT gateway bandwidth.
- Freeze deployments 48 hours before the first shift.
- War room with dashboards for: answer-save p99 latency, event-stream consumer lag, replication lag, ALB 5xx rate, active exam sessions.

Part 5 — Azure implementation

5.1 Service mapping

Layer	AWS	Azure	Notes
DNS	Route 53	**Azure DNS + Traffic Manager**	Traffic Manager for DNS-level failover
CDN + global LB	CloudFront	**Azure Front Door (Premium)**	Combines CDN, global anycast LB and WAF in one service
WAF	AWS WAF	**Azure WAF**	Attaches to Front Door or Application Gateway
DDoS	Shield	**DDoS Protection Standard**	
API gateway	API Gateway	**Azure API Management (APIM)**	Richer policy engine than API Gateway; developer portal included
L7 load balancer	ALB	**Application Gateway**	WAF v2 SKU
L4 load balancer	NLB	**Azure Load Balancer**	Standard SKU
Containers	ECS Fargate	**Azure Container Apps**	Serverless containers, KEDA autoscaling
Kubernetes	EKS	**AKS**	
Serverless	Lambda	**Azure Functions**	Premium plan to avoid cold starts during the exam
Relational	Aurora PostgreSQL	**Azure Database for PostgreSQL — Flexible Server**	Zone-redundant HA; read replicas for scale
Relational (hyperscale)	Aurora	**Cosmos DB for PostgreSQL**	If you need Citus-style sharding
Document	DocumentDB	**Cosmos DB (MongoDB API)**	
Key-value	DynamoDB	**Cosmos DB (NoSQL API)**	Autoscale RU/s; session consistency level
Cache	ElastiCache	**Azure Cache for Redis**	Premium tier for persistence and zone redundancy
Object storage	S3	**Blob Storage**	Hot/Cool/Archive tiers; immutability policies
Streaming	MSK / Kinesis	**Event Hubs**	Kafka-protocol compatible — existing Kafka clients work unchanged
Queue	SQS	**Service Bus**	Sessions, dead-lettering, duplicate detection built in
Identity	Cognito	**Microsoft Entra External ID**	(formerly Azure AD B2C)
Secrets	Secrets Manager	**Azure Key Vault**	
Keys	KMS	**Key Vault / Managed HSM**	Managed HSM for FIPS 140-2 Level 3
Face matching	Rekognition	**Azure AI Face**	**Access is gated** — requires an approved Limited Access application
Malware scan	GuardDuty Malware Protection	**Defender for Storage**	
Monitoring	CloudWatch	**Azure Monitor**	
Tracing	X-Ray	**Application Insights**	Notably strong distributed tracing
IaC	CloudFormation / CDK	**Bicep** (or Terraform)	
Private connectivity	PrivateLink	**Private Link / Private Endpoint**	
Region	ap-south-1 / ap-south-2	**Central India (Pune)** primary, **South India (Chennai)** DR	West India (Mumbai) has limited service availability

5.2 Azure network topology

```
VNet 10.0.0.0/16, three availability zones

Subnet: snet-appgw     10.0.0.0/24
```

```
    -> Application Gateway (delegated subnet, must be dedicated)

Subnet: snet-apim      10.0.1.0/24
    -> API Management, internal VNet mode

Subnet: snet-apps      10.0.10.0/23
    -> Container Apps environment (needs a /23 minimum)

Subnet: snet-data      10.0.20.0/24
    -> Private Endpoints for PostgreSQL, Cosmos DB, Redis, Blob Storage

NSGs on every subnet; ASGs (Application Security Groups) instead of
IP-based rules — the Azure equivalent of referencing security groups.

Private Endpoints give each PaaS service a private IP inside your VNet.
Combined with "public network access = disabled" on the resource, the
database becomes unreachable from the internet entirely.
```

5.3 Meaningful differences from AWS

Azure Front Door does more than CloudFront. It combines CDN, global anycast load balancing, WAF and origin failover in one resource. On AWS you would compose CloudFront + Global Accelerator + WAF. This genuinely simplifies the Azure topology.

APIM's policy engine is more expressive than API Gateway's. Rate limiting, JWT validation, request transformation and caching are all XML policies applied at the global, product, API or operation level. If you are coming from APIGEE, APIM will feel far more familiar than AWS API Gateway does.

Cosmos DB has five consistency levels (Strong, Bounded Staleness, Session, Consistent Prefix, Eventual) as an explicit, tunable setting. DynamoDB gives you eventual or strongly-consistent reads and nothing in between. For exam responses, **Session consistency** is the correct choice — a candidate always reads their own writes, without paying the cost of global strong consistency.

Private Endpoint vs VPC Endpoint. Azure's Private Endpoint injects a NIC with a private IP into your subnet and you disable public access on the resource. AWS's Interface Endpoint is similar, but AWS Gateway Endpoints (S3, DynamoDB) work differently — via route table entries. Azure's model is more uniform.

Entra External ID replaced Azure AD B2C for new tenants. Documentation and older tutorials still say B2C; verify which you are provisioning.

Azure AI Face requires a Limited Access application. Face identification and verification are gated behind an approval process. This is a real project-timeline risk that Rekognition does not have. Plan for it early.

Zone redundancy is often a checkbox rather than an architecture decision. PostgreSQL Flexible Server, Redis Premium, and Container Apps all offer zone redundancy as a configuration flag.

5.4 Azure DR configuration

Component	Strategy	RPO
PostgreSQL Flexible Server	Geo-redundant backup + read replica in Chennai	Minutes (backup) / seconds (replica)
Cosmos DB	Multi-region writes or single-write multi-read	Sub-second
Blob Storage	RA-GZRS + immutability policy + versioning	Minutes
Event Hubs	Geo-disaster recovery pairing (metadata only — **data is not replicated**)	See note
Container Apps	Deployed in both regions, scaled to zero in DR	N/A
Front Door	Origin health probes with automatic failover	Seconds

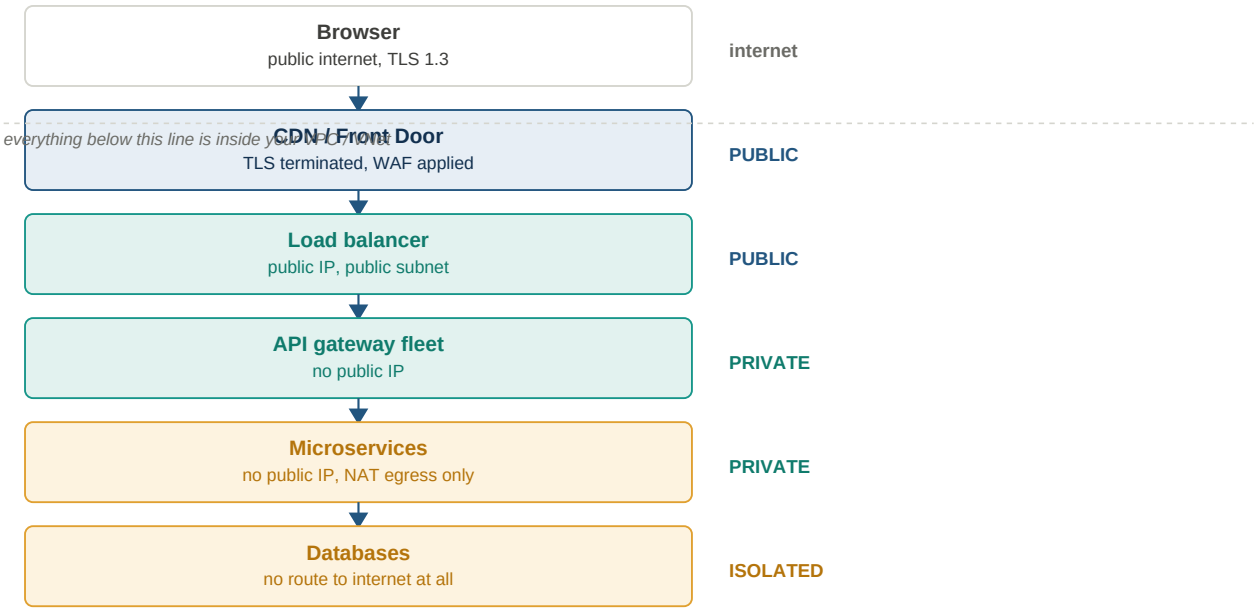
Important caveat: Event Hubs geo-DR replicates *metadata* (namespaces, consumer groups), not message data. If you need message-level replication you must implement it yourself or use Event Hubs on a Premium/Dedicated tier with geo-replication. This is a real difference from MSK Replicator and a common design error.

Part 6 — How the frontend reaches the backend over the internet

This is the question that trips up people who have only worked inside a corporate network.

6.1 The core concept

A browser can only reach endpoints that are publicly resolvable and publicly routable. Your microservices must be neither. The reconciliation is that exactly one tier is public — the edge — and everything behind it is private.



Reachability tiers from browser to database

Three tiers of reachability:

- **Public subnet** — has a route to the internet gateway. Load balancer only.
- **Private subnet** — no inbound from the internet; outbound via NAT for patching and API calls. Application services.
- **Isolated subnet** — no route to NAT or the internet gateway. Databases. Even if compromised, a process here cannot exfiltrate data directly to the internet.

6.2 DNS and TLS

```
www.exam.gov.in   ALIAS -> CDN distribution
api.exam.gov.in   ALIAS -> CDN distribution (different origin behaviour)
```

- Certificate issued by ACM (AWS) or App Service Certificate / Key Vault (Azure), attached to the edge.
- **Re-encrypt to origin.** Edge-to-origin traffic must also be TLS. "TLS to the edge, HTTP to origin" is a common and serious misconfiguration.
- HSTS with a long max-age, and preload if the domain is dedicated.
- The load balancer's certificate can be internal, but it must exist.

6.3 CORS — the thing that actually bites people

If your SPA is served from `www.exam.gov.in` and calls `api.exam.gov.in`, **those are different origins** and every non-simple request triggers a preflight `OPTIONS`.

Recommended solution: avoid CORS entirely with same-origin path routing.

```
https://exam.gov.in/          -> SPA static assets (CDN -> object storage)
https://exam.gov.in/api/*     -> API (CDN -> load balancer -> gateway)
```

One origin. No preflight. No credential complications. One fewer thing to misconfigure in production. **This is the answer I would give in an interview** — the question is usually testing whether you reach for a workaround or eliminate the problem.

If you genuinely need cross-origin:

```
Access-Control-Allow-Origin:      https://www.exam.gov.in   (specific, never *)
Access-Control-Allow-Credentials: true
Access-Control-Allow-Methods:     GET, POST, PUT, DELETE
Access-Control-Allow-Headers:     Authorization, Content-Type, X-Request-Id
Access-Control-Max-Age:           600
```

Rules:

- `Access-Control-Allow-Origin: *` **cannot** be combined with `Allow-Credentials: true`. The browser rejects it.
- Handle `OPTIONS` at the gateway, not in every service.
- `Max-Age` caches the preflight so you are not doubling every request.

6.4 Authentication token handling

Approach	XSS risk	CSRF risk	Complexity	Verdict
JWT in `localStorage`	**High** — any XSS steals the token	None	Low	Common, but wrong for high-stakes systems
JWT in `httpOnly; Secure; SameSite=Strict` cookie	Low	Needs CSRF token	Medium	Good
BFF pattern — session cookie to a backend-for-frontend that holds the token server-side	**None** — token never reaches the browser	Handled by BFF	Higher	**Best for an exam portal**

Token flow: OAuth 2.0 / OIDC **Authorization Code with PKCE**. The implicit flow is deprecated and should never appear in a new design — saying so unprompted is a good interview signal.

```
1. Browser -> /authorize (with code_challenge)
2. User authenticates at the identity provider
3. Redirect back with an authorization code
4. BFF exchanges code + code_verifier for tokens  (server-to-server)
5. BFF stores tokens, sets an httpOnly session cookie
6. Browser sends only the session cookie thereafter
7. BFF attaches the access token to downstream calls
```

6.5 Real-time channels

The exam client needs a server-authoritative timer and heartbeat.

Option	Use when
WebSocket	Bidirectional need. Heartbeat + server push. ALB and Application Gateway both support it.
Server-Sent Events	Server-to-client only. Simpler, auto-reconnects, works over plain HTTP/2. Often sufficient.
Long polling	Fallback only

For the exam timer, SSE plus ordinary POSTs for answer saves is usually enough and is markedly simpler to operate than WebSocket at scale. Reach for WebSocket only if you need genuine bidirectional low-latency messaging.

Do not put the timer in the client. The client renders a countdown; the server holds the truth and every heartbeat re-synchronises it.

6.6 Resilience at the client

For an exam client specifically:

- **Prefetch the whole encrypted paper** before the shift starts. Decryption key released at start time. Network failure mid-exam then costs nothing.
- **IndexedDB as a write-ahead log.** Every answer lands locally first, then syncs. Survives a browser crash or tab close.
- **Queue and replay on reconnect,** carrying `client_seq` so out-of-order retries cannot overwrite newer answers.
- **Service worker** to serve the app shell offline.
- **Exponential backoff with jitter** on retries. Without jitter, 125,000 clients reconnecting simultaneously after a blip will produce a thundering herd that takes down the service you just recovered.

6.7 Security layers, outermost to innermost

Layer	Control
Edge	DDoS protection, WAF (OWASP rules + rate-based), geo-restriction to India
CDN	Signed URLs for private objects, origin access control
Gateway	JWT validation, per-consumer rate limiting, request schema validation
Network	Public/private/isolated subnets, security groups referencing each other
Service	mTLS between services if a mesh is present, least-privilege IAM roles
Data	Encryption at rest with customer-managed keys, encryption in transit, field-level encryption for the most sensitive attributes
Audit	Immutable WORM log of every state transition

Compliance for an Indian examination body:

- **DPDP Act 2023** applies. Photos and signatures are personal data; face data used for matching should be treated as biometric — explicit consent, stated purpose, defined retention, deletion on expiry.
- **Data residency** — keep everything in Indian regions. Both AWS and Azure have two each, which makes in-country DR straightforward.
- **If Aadhaar is in scope**, UIDAI regulations are considerably stricter than DPDP. The number cannot be stored in plaintext; store a hash plus the last four digits, or avoid storing it and retain only the authentication response.
- **Audit every access to a candidate's photo** — who, when, from where.

Part 7 — Interview questions and answers

Java

Q: Why is HashMap capacity always a power of two? So the bucket index can be computed as `hash & (capacity - 1)` instead of `hash % capacity`. Bitwise AND is dramatically cheaper than integer division. It also makes the resize optimisation possible: after doubling, an entry either stays at index `i` or moves to `i + oldCap`, decided by a single bit test.

Q: What does `h ^ (h >>> 16)` accomplish? It mixes the high 16 bits of the hash into the low 16. Because the index only uses the low bits, a `hashCode()` that varies only in its high bits would otherwise collide catastrophically. `Integer.hashCode()` returns the value itself, so small integers already distribute well — but string and object hashes often do not.

Q: What happens if a key's hashCode changes after insertion? The entry becomes unreachable. `get` computes a new index, looks in the wrong bucket, and returns null. The entry is still consuming memory and will still be rehashed on resize, but you can never retrieve it. This is why map keys should be immutable.

Q: Why did Java 8 add treeification? Two reasons. Nominally, to cap worst-case lookup at O(log n) instead of O(n). Actually, to defend against hash-collision denial of service — an attacker who can control the keys you insert could otherwise force every entry into one bucket and turn every operation into a linear scan.

Q: Why is ConcurrentHashMap necessary — what specifically breaks? Beyond ordinary lost updates: in the pre-Java-8 implementation, concurrent resizing could produce a cycle in a bucket's linked list, causing `get()` to spin forever at 100% CPU. Java 8 changed resize to preserve order and avoid this, but the map is still not thread-safe for compound operations.

Load balancing and gateways

Q: Does an API Gateway replace a load balancer? Only if the gateway is a managed service. A self-hosted gateway is itself a fleet of processes and cannot load balance itself — you need an L7 balancer in front of it. The deeper point is that they solve different problems: a load balancer chooses *which instance*, a gateway decides *which service and whether you are allowed*.

Q: ALB or NLB? ALB when you need to route on path, host or header, terminate TLS, or handle gRPC and WebSocket. NLB when you need extreme throughput, static IPs, source IP preservation, or non-HTTP protocols. NLB is also the right choice as an entry point for a service mesh ingress.

Q: Why is least-outstanding-requests usually better than round robin? Round robin assumes every request costs the same. In reality one request might be a cached lookup and the next a report generation. Round robin will happily send a new request to an instance already saturated with slow work. Least-outstanding-requests naturally routes away from instances that are struggling.

Q: What breaks if you forget connection draining? Every deployment drops in-flight requests. The instance is deregistered and killed while it is still processing responses. Users see 502s during every release, which teams often misdiagnose as an application bug.

Rate limiting

Q: Why token bucket over fixed window? Fixed window has a boundary exploit: with a 100/minute limit, a client can send 100 requests at 11:59:59 and 100 more at 12:00:00 — 200 requests in one second, all "within limit." Token bucket has no boundaries; it refills continuously.

Q: Why does the Redis implementation need Lua? Atomicity. Read-modify-write over separate round trips is a race — two concurrent requests both read "1 token remaining" and both proceed. Redis executes a Lua script atomically, so the check-and-decrement cannot interleave.

Q: Should a rate limiter fail open or closed? Open. If Redis is unavailable, the correct behaviour is to allow traffic rather than reject everything — the rate limiter should never be the cause of an outage. Authentication is the opposite and must fail closed. Being able to articulate *why* these differ is the point of the question.

Q: What is wrong with rate limiting by IP? Carrier-grade NAT means thousands of mobile users can share one address, so a limit meant for one abuser blocks a whole region. Conversely, an attacker with a botnet or an IPv6 allocation has effectively unlimited addresses. Use authenticated identity where you have it, and for IPv6 limit on the `/64` prefix rather than the individual address.

Storage and data

Q: Why not store images in the database? Backup and restore times balloon, every read replica carries the full binary weight, storage costs roughly 5× more per GB than object storage, you cannot serve through a CDN, and streaming large objects ties up connection-pool slots. Store the key and metadata in the database; put the bytes in object storage.

Q: When would you actually put a blob in the database? When it is small (a few KB), always read together with the row, and you need it inside the same transaction — for example a stored signature hash or a small encrypted token. Even then, weigh it carefully.

Q: Why presigned URLs rather than proxying uploads? At 3 million uploads, proxying means hundreds of gigabytes flowing through application servers, occupying request threads for the duration of each transfer, for no benefit. Presigned URLs let the client write directly to object storage while the scope, content type, size limit and expiry are still fully under your control.

Q: Document store or relational for candidate registration? Relational. Registration is the most transactional part of the system — payment confirmation, application-number generation and slot allocation must be atomic, and duplicate prevention wants a real unique constraint. The document store belongs on the *question bank*, where the schema genuinely varies by question type.

Disaster recovery

Q: Is Multi-AZ disaster recovery? No. Multi-AZ is high availability within one region. It protects against a datacentre failure. It does not protect against a regional control-plane outage, a bad deployment, an accidental data deletion, or a compromised account. Conflating the two is the most common DR mistake.

Q: How do you choose between pilot light and warm standby? By RTO and by what an hour of downtime costs. Pilot light means tens of minutes to hours of recovery at ~15% standby cost; warm standby means minutes at ~50%. For a national exam I would run pilot light for most of the year and switch to warm standby for the exam window — the incremental cost for twelve days is trivial against the cost of re-conducting the exam.

Q: What is the most common reason a DR failover fails on the first real test? Regional service quotas and encryption key permissions. The data replicated perfectly for two years, and then the DR region refused to launch the instances because the account quota was at default, or the encrypted snapshot could not be decrypted because the key grant in the destination region was never created.

System design

Q: What is the hardest part of an online examination system? Not throughput — 6,000 writes per second is modest. It is that you cannot retry. If the system fails during a three-hour window, those candidates' exams must be re-conducted at national scale. So the design optimises for durability and graceful degradation over efficiency: prefetch the whole paper so network failure costs nothing, write answers locally before sending them, publish every response to an immutable log, and make the timer server-authoritative.

Q: How do you handle 125,000 candidates hitting results at once? Do not compute anything on request. Evaluation is a batch job that runs after the objection window; each score card is rendered to a static object; the CDN is warmed before the announcement. The spike hits edge caches and the database sees essentially nothing. Trying to serve that load dynamically is the classic failure.

Q: How do you prevent a candidate from losing answers on a network drop? Local-first writes. Every answer lands in IndexedDB before the network request, so a browser crash loses nothing. Writes are queued and replayed on reconnect, each carrying a monotonic `client_seq` so a delayed retry cannot overwrite a newer answer. The whole paper is prefetched at start, so the candidate can keep working through a total network outage.

Q: Why put an event stream in front of the database for exam responses? It makes the durable log the source of truth and the database a projection. Disaster recovery becomes "replay from offset" rather than "hope replication lag was low." For a system where a lost answer is a legal liability rather than a bug, that is a materially stronger guarantee. The cost is that consumers must be genuinely idempotent, and offsets do not translate across replicated clusters.

Q: AWS or Azure for this — does it matter? The architecture is identical; only the service names change. The differences that would actually influence a decision: Azure Front Door collapses CDN, global load balancing and WAF into one resource where AWS needs three; Cosmos DB's five explicit consistency levels give finer control than DynamoDB's two; APIM's policy engine is more expressive than API Gateway's and will feel familiar to an APIGEE team. Against that, Azure AI Face requires an approval process that Rekognition does not, which is a genuine schedule risk. In practice, existing team skills and enterprise

agreements decide it.

Quick reference — the numbers worth memorising

Quantity	Value
HashMap default capacity / load factor	16 / 0.75
Treeify / untreeify threshold	8 / 6
L1 cache reference	~1 ns
Main memory reference	~100 ns
SSD random read	~150 µs
Round trip within a datacentre	~0.5 ms
Round trip Mumbai to Chennai	~30–40 ms
Round trip India to US East	~200–250 ms
Disk seek (spinning)	~10 ms
Single Redis instance	~100k ops/s
Single PostgreSQL primary, well-tuned	~10–20k writes/s
API Gateway REST pricing	~$3.50 / million requests
API Gateway HTTP pricing	~$1.00 / million requests

End of notes. Verify all cloud service names, quotas and pricing against current provider documentation before relying on them.